

Ferramentas open source de SAST SCA IaC Security e DAST no pipeline DevSecOps

Análise do Grupo 4 com Semgrep OWASP Dependency Check Checkov e OWASP ZAP

SAST	SCA	IaC	DAST
Semgrep	Dependency Check	Checkov	OWASP ZAP

Disciplina

DevSecOps preparatório E|CDE

Turma e professor

2TDCPF · Fabio Pires

Líder técnico

Tiago Louzã

IntegrantesLeando de Souza Silva · Luiz Fernando ·
Fabricio de Freitas Evangelista · Erik Gunnar**Local e ano**

São Paulo 2026

DADOS DO TRABALHO

Identificação do trabalho

Campo	Informação
Tema	Ferramentas open source de SAST, SCA, IaC Security e DAST
Ferramentas do Grupo 4	Semgrep, OWASP Dependency-Check, Checkov e OWASP ZAP
Alvo autorizado	OWASP Juice Shop v20.2.0 em ambiente Docker local
Laboratório principal	Semgrep e OWASP ZAP, categorias diferentes
Execuções complementares	Dependency-Check e Checkov com relatórios versionados
Política de gate	Bloqueio de achados HIGH e CRITICAL
Data da validação	15 de setembro de 2026

Síntese das execuções

Ferramenta	Categoria	Situação	Resultado principal
Semgrep	SAST	Concluído	69 achados no scan geral; red com 1 HIGH e green sem achados
Dependency-Check	SCA	Inconclusivo	15 dependências, 0 CVEs e cobertura reduzida por ausência de lockfiles
Checkov	IaC	Concluído	42 controles aprovados e 24 reprovados no Terraform oficial
OWASP ZAP	DAST	Concluído	11 tipos de alerta, com 1 HIGH; gate bloqueado

Todos os resultados foram mantidos em formatos legíveis e processáveis por máquina. Os relatórios JSON, SARIF e HTML permitem reproduzir a análise, revisar a classificação e demonstrar o gate mesmo quando o ambiente ao vivo não estiver disponível.

VISÃO EXECUTIVA

Resumo

Este relatório compara Semgrep, OWASP Dependency-Check, Checkov e OWASP ZAP no pipeline DevSecOps e registra a execução das quatro ferramentas atribuídas ao Grupo 4. O alvo foi o OWASP Juice Shop v20.2.0, aplicação deliberadamente vulnerável e autorizada para treinamento. O ambiente foi isolado com Docker e os resultados foram preservados no repositório para auditoria e apresentação.

O Semgrep foi executado primeiro com a configuração geral da comunidade, sem regra própria. O scan percorreu 1.027 arquivos, aplicou 408 regras e reportou 69 achados. A análise encontrou SQL Injection nas rotas de login e busca. Para a demonstração red e green, uma regra local mais estreita foi mantida apenas para garantir tempo e resultado determinísticos: o código vulnerável produziu um HIGH, a consulta foi parametrizada com replacements do Sequelize e o mesmo teste passou sem achados.

O Dependency-Check terminou em sete segundos, identificou 15 dependências e não reportou CVEs. Esse zero não foi tratado como evidência de segurança, pois o projeto analisado não continha os lockfiles nem node_modules necessários para um inventário completo, e quatro arquivos de teste geraram exceções. O Checkov, por sua vez, aplicou políticas nativas ao Terraform oficial do Juice Shop: 42 verificações passaram e 24 falharam, com problemas de exposição pública, HTTP sem redirecionamento para HTTPS, ausência de logs e falta de WAF.

O ZAP executou spiders e active scan apenas contra o contêiner local. Foram descobertas 234 URLs pelo AJAX spider e registrados 11 tipos de alerta em 39 instâncias. A SQL Injection HIGH foi correlacionada com a linha de código identificada pelo Semgrep. O gate bloqueou os cenários vulneráveis e aprovou o recorte SAST corrigido. O resultado confirma que SAST, SCA, IaC Security e DAST respondem a perguntas diferentes e precisam coexistir, com critérios de cobertura e triagem explícitos.

Palavras-chave: DevSecOps; SAST; SCA; IaC Security; DAST; Semgrep; Dependency-Check; Checkov; OWASP ZAP; quality gate.

NAVEGAÇÃO

Sumário

Seção	Página
1 Introdução	5
2 Fundamentos e posicionamento no pipeline	6
3 Metodologia do laboratório	8
4 Semgrep	9
5 OWASP Dependency Check	14
6 Checkov	17
7 OWASP ZAP	20
8 Análise dos achados	24
9 Comparação e escolha do toolchain	26
10 Integração contínua e quality gate	27
11 Limitações e conclusão	28
Referências	30
Apêndices	31

A numeração corresponde ao PDF validado. Os apêndices reúnem os comandos e o mapa completo das evidências versionadas.

ESCOPO E OBJETIVOS

1 Introdução

DevSecOps incorpora verificações de segurança ao desenvolvimento para reduzir o intervalo entre a introdução de uma falha e sua correção. O objetivo não é acumular scanners, mas posicionar controles com entradas, saídas e responsáveis definidos. O NIST SSDF recomenda que a organização prepare pessoas e processos, proteja o software, produza componentes seguros e responda a vulnerabilidades durante todo o ciclo [1].

O trabalho do Grupo 4 cobre quatro frentes complementares. O Semgrep analisa o código escrito pela equipe; o Dependency-Check procura vulnerabilidades conhecidas nas dependências; o Checkov avalia configurações de infraestrutura como código; e o ZAP interage com a aplicação em execução. As quatro ferramentas são open source, automatizáveis e capazes de produzir resultados para integração contínua, mas operam sobre evidências diferentes.

Objetivo geral

Avaliar tecnicamente as quatro ferramentas atribuídas ao Grupo 4, executar cada uma em um alvo autorizado e demonstrar como os resultados podem interromper ou permitir a progressão de uma entrega.

Objetivos específicos

- Diferenciar SAST, SCA, IaC Security e DAST quanto à entrada, técnica, momento de execução e limitações.
- Registrar identificação, manutenção, licença, comunidade, instalação, formatos e integrações de cada ferramenta.
- Executar as quatro ferramentas e interpretar resultados, inclusive quando a cobertura for insuficiente.
- Demonstrar build red e green com Semgrep e um bloqueio DAST com OWASP ZAP.
- Classificar três achados como verdadeiro positivo ou falso positivo e propor correções verificáveis.

Questão de pesquisa

Como um conjunto de ferramentas open source pode ampliar a cobertura de segurança sem transformar a pipeline em uma coleção de alertas sem contexto? A resposta adotada é combinar controles em camadas, preservar os relatórios originais e aplicar gates apenas quando a equipe consegue explicar o limiar, a cobertura e o tratamento de exceções.

SAST SCA IAC E DAST

2 Fundamentos e posicionamento no pipeline

As categorias não competem pelo mesmo problema. SAST observa o código próprio antes da execução. SCA constrói um inventário de componentes de terceiros e correlaciona versões com vulnerabilidades publicadas. IaC Security interpreta arquivos declarativos que criarão recursos. DAST envia requisições a um sistema ativo e avalia respostas. A diferença entre SAST e SCA é especialmente importante: uma falha no código da equipe requer alteração de implementação; uma vulnerabilidade em dependência exige atualizar, substituir, remover ou mitigar um componente.

Categoria	Pergunta respondida	Entrada principal	Aplicação ativa	Estágio
SAST	Existe padrão inseguro no código próprio?	Código-fonte	Não	Code e Build
SCA	Há componente conhecido como vulnerável?	Manifesto, lockfile e SBOM	Não	Build
IaC	A configuração provisionará recurso inseguro?	Terraform e manifestos	Não	Code, Build e Deploy
DAST	O comportamento expõe falha acessível?	Aplicação em execução	Sim	Test e Release

Shift left sem eliminar o DAST

Shift left antecipa feedback. Semgrep, Dependency-Check e Checkov podem rodar antes de um deploy, quando a correção tende a ser mais simples. Essa antecipação não substitui o ZAP. Um scanner estático não observa cabeçalhos emitidos pelo servidor, composição real de rotas, autenticação de sessão, proxies, variáveis de ambiente ou comportamento integrado. O DAST verifica o que ficou acessível após a montagem do sistema [2].

Cobertura em camadas

Uma pipeline eficiente separa verificações rápidas de verificações profundas. Regras locais e políticas de IaC podem executar em cada pull request. SCA completo exige inventário e atualização de bases. DAST precisa de um ambiente implantado e de tempo para descoberta. O gate deve considerar essas diferenças para não interpretar ausência de achados como cobertura total.

INVENTÁRIO E RASTREABILIDADE

SBOM e relação com SCA

Uma Software Bill of Materials é uma lista estruturada de componentes, versões, fornecedores, identificadores e relações. Ela não é um scanner por si só. A SBOM fornece o inventário que permite perguntar onde um componente vulnerável aparece e quais produtos precisam de resposta. A CISA destaca elementos mínimos como autor do documento, fornecedor, nome, versão, identificadores, relações e data [3].

O Dependency-Check pode complementar esse inventário ao tentar identificar componentes e associá-los a CPEs e CVEs. O CycloneDX fornece formatos padronizados para troca de SBOM [21]. A qualidade do SCA depende da qualidade do inventário: sem lockfile, a versão resolvida pode ser desconhecida; sem dependências instaladas, analisadores de ecossistema podem perder componentes; sem atualização da base, uma vulnerabilidade recente pode não aparecer.

Artefato	O que registra	Uso no pipeline	Limitação
Manifesto	Dependências declaradas	Instalação e resolução	Pode permitir intervalos de versão
Lockfile	Versões efetivamente resolvidas	Reprodutibilidade e SCA	Precisa estar atualizado
SBOM	Componentes e relações	Inventário e resposta	Não prova explorabilidade
Relatório SCA	Correspondências com vulnerabilidades	Triagem e gate	Pode ter falsos positivos e negativos

Critério para gates

O gate deste laboratório usa severidade HIGH como limiar por ser simples de demonstrar. Em produção, a decisão precisa acrescentar confiança, alcance, exposição,

existência de correção e validade da exceção. O resultado original permanece arquivado; a normalização serve para decidir a pipeline, não para apagar detalhes do scanner.

AMBIENTE E CRITÉRIOS

3 Metodologia do laboratório

O laboratório foi executado em Windows com Docker Desktop. O alvo foi a release v20.2.0 do OWASP Juice Shop, autorizada pelo enunciado. O código oficial foi mantido em target/juice-shop. Uma cópia pontual de routes/search.ts foi criada em target/juice-shop-fixed para demonstrar a correção sem alterar a referência vulnerável usada pelo ZAP.

Escopo ético

Todos os testes ativos apontaram para o contêiner local do Juice Shop. O serviço foi publicado apenas em 127.0.0.1:3000 e o ZAP acessou o alvo pela rede interna do Compose. Nenhum domínio público, sistema de terceiros ou ambiente corporativo foi varrido. Essa restrição é parte do método, não apenas uma precaução operacional.

Sequência de execução

1. Executar Semgrep com --config auto sobre o repositório completo e preservar JSON e log.
2. Executar o cenário SAST red no arquivo vulnerável e confirmar que o gate bloqueia o HIGH.
3. Aplicar a consulta parametrizada, repetir o scan e confirmar o cenário green.
4. Executar Dependency-Check e registrar vulnerabilidades, exceções e limitações do inventário.
5. Executar Checkov sobre o Terraform oficial com políticas nativas.
6. Subir o Juice Shop, executar o plano de automação do ZAP e avaliar o JSON no mesmo gate.

Ferramenta	Versão executada	Escopo	Tempo observado
Semgrep	1.172.0	1.027 arquivos no geral; 1 arquivo no red e green	cerca de 3 min no geral
Dependency-Check	13.0.0	Juice Shop e exemplo SCA	7 s
Checkov	3.3.11	26 recursos Terraform	cerca de 9 s
OWASP ZAP	2.17.0	Juice Shop local	active scan 3 min 48 s

Os tempos são medições do ambiente do grupo e não benchmarks universais. Cache de imagens, desempenho do host, quantidade de arquivos e atualização de bases alteram o resultado.

SAST

4 Semgrep

Identificação e atividade do projeto

Atributo	Informação
Mantenedor	Semgrep Inc. e comunidade
Licença do motor open source	LGPL 2.1
Início público	2017
Linguagens de implementação	OCaml no motor e Python na CLI
Comunidade em 15 set. 2026	aprox. 16,7 mil estrelas, 1,1 mil forks e 10.297 commits
Release atual na consulta	v1.177.0, publicada em 10 set. 2026
Versão do laboratório	v1.172.0

O repositório oficial descreve Semgrep Community Edition como um mecanismo open source de análise estática e informa suporte a mais de 30 linguagens [5]. A sucessão de releases entre julho e setembro de 2026 indica manutenção frequente [7]. A edição comunitária cobre análise local; recursos proprietários acrescentam análise entre arquivos, regras comerciais e triagem hospedada.

Fundamento técnico

Semgrep transforma o código em uma representação sintática e compara essa estrutura com padrões semelhantes à própria linguagem. Isso evita depender apenas de texto ou expressão regular. As regras podem usar pattern matching, metavariáveis e, em modos compatíveis, análise de taint para representar origem, propagação, sanitização e destino de dados.

A ferramenta detecta APIs perigosas, injeções, configurações inseguras, uso incorreto de bibliotecas e padrões específicos da equipe quando existe uma regra aplicável. A Community Edition não entende toda a semântica de execução e possui limites em fluxos entre funções e arquivos. Reflexão, geração dinâmica de código e regras ausentes podem produzir falsos negativos. Um match sintático também pode ser irrelevante quando aparece em teste, exemplo ou código inalcançável.

SEMGREP NA PRÁTICA

Instalação configuração e integração

Instalação e formatos

O Semgrep pode ser instalado por pip, pipx, Homebrew e imagem Docker. O laboratório usou Docker para evitar dependências locais e fixar a versão. As saídas mais úteis para automação são JSON e SARIF; o terminal serve à execução guiada. O SARIF pode ser enviado ao GitHub Code Scanning, enquanto o JSON permite aplicar uma política própria.

```
docker run --rm -v "${ProjectDir}target/juice-shop:/src/target/juice-shop" \
-v "${ProjectDir}reports:/src/reports" semgrep/semgrep:1.172.0 \
semgrep scan --config auto --json \
--output /src/reports/semgrep-auto.json /src/target/juice-shop
```

Uso geral sem regra própria

A opção `--config auto` selecionou regras comunitárias compatíveis com as linguagens detectadas. Portanto, o scan geral não dependeu de uma regra criada pelo grupo. Essa é a forma mais próxima de um uso geral: ela oferece amplitude, mas também inclui achados de manutenção, exemplos, workflows e infraestrutura que exigem triagem.

Regra local do gate

A regra em `security/semgrep.yml` não foi necessária para encontrar SQL Injection no scan geral. Ela foi mantida para o laboratório de 12 minutos, porque limita o escopo a uma construção conhecida e torna o red e green reproduzível. A metadata `devsecops_severity`

normaliza o achado como HIGH e CWE-89. Em produção, regras locais devem ter testes positivos e negativos, responsável e revisão quando o framework mudar.

Baselines supressão IDE e pre commit

Semgrep permite ignorar trechos com nosemgrep, excluir caminhos, comparar com um commit de baseline e executar somente achados novos em CI. Supressões devem registrar justificativa e prazo; esconder um diretório inteiro reduz cobertura. Extensões de IDE e hooks de pre-commit oferecem feedback antes do push. Para repositórios grandes, recomenda-se um conjunto rápido no pre-commit e uma política mais ampla no CI [6].

SEMGREP SEM REGRA PERSONALIZADA

Execução geral e evidência de terminal

O scan geral considerou 1.074 regras comunitárias e executou 408 delas sobre 1.027 arquivos. Foram produzidos 69 achados: 18 ERROR, 46 WARNING, 2 MEDIUM e 3 INFO. O relatório também registrou 53 erros ou avisos de análise, em grande parte associados a snippets deliberadamente incompletos da aplicação de treinamento. A taxa de linhas analisadas foi próxima de 99,9%.

```

>> semgrep scan `
>> --config auto `
>> --json
>> --output /src/reports/semgrep-auto.json `
>> /src/target/juice-shop 2>&1 |
>> Tee-Object ".\reports\semgrep-auto-live.log"
docker :
No linha:1 caractere:1
+ docker run --rm `
+ docker run --rm `
+ CategoryInfo          : NotSpecified: (                :String) [], RemoteException
+ FullyQualifiedErrorId : NativeCommandError

Scan Status

Scanning 1027 files tracked by git with 1074 Code rules:

Language    Rules    Files    Origin    Rules
-----
<multilang> 77       685     Community 1074
ts           163      507
json         4        105
yaml         35       98
html         1        82
solidity     21       17
js           153      12
terraform    101      11
dockerfile   6        2
bash         4        1

Scan Summary

✓ Scan completed successfully.
• Findings: 69 (69 blocking)
• Rules run: 408
• Targets scanned: 1027
• Parsed Lines: ~99.9%
• Scan skipped:
  • Files larger than files 1.0 MB: 2
  • Files matching .semgrepignore patterns: 7
• For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 408 rules on 1027 files: 69 findings.
A new version of Semgrep is available. See https://semgrep.dev/docs/upgrading
Too many findings? Try Semgrep Pro for more powerful queries and less noise.
See https://sg.run/false-positives.

```

Figura 1 Execução real do Semgrep com configuração auto.
Fonte: captura de terminal do Grupo 4, 2026.

A mensagem NativeCommandError no início da captura foi produzida pelo PowerShell 5.1 ao converter a saída stderr do processo Docker em um registro de erro durante o redirecionamento 2>&1. Ela não representa falha do scanner: a mesma captura mostra Scan completed successfully e o JSON foi gravado. Para uma apresentação mais limpa, o comando pode ser executado sem o redirecionamento combinado.

SEMGREP NO QUALITY GATE

Resultados red e green correção e avaliação crítica

Resultado determinístico

Cenário	Arquivos	Regras	Achados	Decisão
Red	1	3	1 HIGH	Gate bloqueado
Green	1	3	0	Gate aprovado

Código vulnerável

```
models.sequelize.query(`SELECT * FROM Products WHERE
((name LIKE '${criteria}%' OR description LIKE '${criteria}%')
AND deletedAt IS NULL) ORDER BY name`)
```

Correção aplicada

```
models.sequelize.query(`SELECT * FROM Products WHERE
((name LIKE :criteria OR description LIKE :criteria)
AND deletedAt IS NULL) ORDER BY name`,
{ replacements: { criteria: `${criteria}%` } })
```

A correção separa o valor fornecido pelo usuário da estrutura da consulta. O marcador `:criteria` é resolvido pelo driver com `replacements`, reduzindo a possibilidade de o conteúdo alterar a sintaxe SQL. A validação `green` prova que a regra local deixou de encontrar o padrão naquele arquivo; não prova que todo o Juice Shop está seguro.

Falsos positivos e desempenho

As 69 ocorrências não foram integralmente classificadas, portanto não existe uma taxa geral confiável. No recorte da regra comunitária de injeção via Sequelize, seis matches foram encontrados: dois em rotas executáveis e quatro em `data/static/codefixes`, arquivos didáticos. Para o risco de produção, esses quatro são ruído operacional, o que corresponde a 66,7% no recorte. A regra local do laboratório produziu um match verdadeiro e nenhum falso positivo. O scan geral levou cerca de três minutos; o recorte de um arquivo terminou em menos de um segundo.

Ponto forte: criação rápida de regras e feedback próximo ao código. Limitação: a cobertura depende do conjunto de regras e do contexto disponível. Cenário ideal: revisão de pull request, IDE e gates de padrões que a equipe consegue testar. Alternativas incluem OpenGrep, CodeQL e SonarQube; o Semgrep foi escolhido pela execução local simples, regras legíveis e bom encaixe no laboratório.

SCA

5 OWASP Dependency Check

Identificação e atividade do projeto

Atributo	Informação
Mantenedor	OWASP Dependency-Check, Jeremy Long e comunidade
Licença	Apache 2.0
Ano de criação	2012
Linguagem principal	Java
Comunidade em 15 set. 2026	aprox. 7,7 mil estrelas, 1,4 mil forks e 11.258 commits
Release atual e executada	v13.0.0, publicada em 3 ago. 2026

Dependency-Check é um utilitário de Software Composition Analysis que tenta identificar componentes e associá-los a CPEs para consultar vulnerabilidades publicadas [8]. A versão 13.0.0 estava atual na data do laboratório [10]. O projeto mantém CLI e integrações com Maven, Gradle, Ant e Jenkins.

Fundamento técnico

A ferramenta usa analisadores por tipo de arquivo e ecossistema para coletar evidências de nome, fornecedor e versão. Em seguida, tenta resolver identificadores de plataforma e consulta dados de vulnerabilidade, principalmente do NVD. A correspondência não é uma prova automática de explorabilidade. Nomes genéricos, componentes empacotados e metadados incompletos podem produzir CPEs errados ou deixar de identificar a versão real.

O Dependency-Check detecta CVEs conhecidos quando consegue identificar o componente. Ele não encontra uma falha inédita no código próprio, não verifica configuração de nuvem e não ataca a aplicação. Também não substitui análise de alcance: uma biblioteca pode estar presente sem que a função vulnerável seja chamada. O sentido inverso também ocorre, pois um inventário incompleto gera falso negativo.

Bases e atualização

A documentação recomenda chave de API do NVD e cache para CI, pois downloads completos e limites de requisição podem aumentar o tempo [8]. O laboratório usou uma

imagem já disponível e `--noupdate` para evitar dependência de rede. Essa decisão tornou a execução rápida, mas vinculou o resultado à base existente na imagem. Em produção, a base precisa de atualização controlada e monitorada.

DEPENDENCY CHECK NA PRÁTICA

Instalação uso integração e tratamento de exceções

Formas de execução

A ferramenta pode ser executada por CLI, Docker, Homebrew, Maven, Gradle, Ant e Jenkins. Os formatos incluem HTML, JSON, XML, CSV, JUnit, SARIF, GitLab e Jenkins. O parâmetro `failOnCVSS` ou seu equivalente no plugin permite falhar o build por pontuação, mas deve ser combinado com supressões revisadas e cobertura comprovada.

```
dependency-check --scan /src --out /report --format ALL `
--project "Checkpoint DevSecOps Grupo 4" `
--noupdate --disableOssIndex --enableExperimental
```

Supressões e baselines

Falsos positivos podem ser tratados com um arquivo XML de suppression que registra CVE, CPE, pacote ou hash. Uma supressão útil inclui motivo e data de expiração. A equipe também pode comparar resultados com um baseline aprovado e bloquear apenas novas vulnerabilidades acima do limiar. Suprimir porque o build precisa passar, sem documentar alcance ou mitigação, remove o controle.

Integração

Maven e Gradle permitem executar a análise no ciclo de build; Jenkins possui plugin; a CLI funciona em GitHub Actions, GitLab CI e outros runners. O SARIF pode alimentar Code Scanning e plataformas que aceitam o padrão. O DefectDojo possui importadores para relatórios de Dependency-Check. A ferramenta não tem o mesmo encaixe de um linter em IDE ou pre-commit: a atualização de bases e a resolução de dependências tornam o CI ou um job agendado mais apropriado.

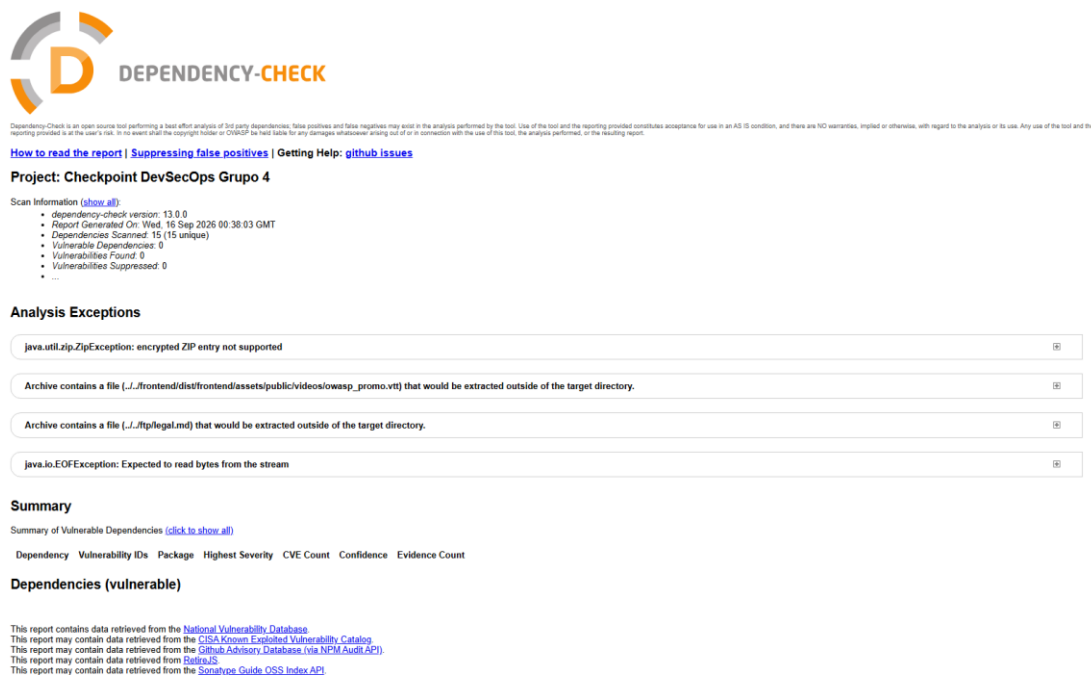
Dependência do inventário

Para Node.js, o lockfile e o diretório de dependências ajudam a determinar versões efetivas. O enunciado pede interpretação crítica, portanto um relatório vazio precisa ser confrontado com os avisos de cobertura. Esse ponto foi decisivo na execução do grupo.

DEPENDENCY CHECK EXECUTADO

Resultado observado e interpretação

A versão 13.0.0 analisou 15 dependências em sete segundos e reportou zero vulnerabilidades. Quatro exceções apareceram ao abrir arquivos propositalmente malformados ou ofensivos usados pelos testes do Juice Shop: dois ZIPs com tentativa de path traversal, um ZIP criptografado e um executável truncado. Esses erros não são CVEs do projeto; são limitações do processamento desses arquivos.



The screenshot shows the OWASP Dependency-Check report interface. At the top, there's a logo and the text 'DEPENDENCY-CHECK'. Below it, a disclaimer states that the tool is an open-source tool performing a best-effort analysis. The report title is 'Project: Checkpoint DevSecOps Grupo 4'. Under 'Scan Information', it shows the version is 13.0.0, the report was generated on Wed, 16 Sep 2026 00:38:03 GMT, 15 dependencies were scanned (15 unique), and 0 vulnerabilities were found or suppressed. The 'Analysis Exceptions' section lists four items: 'java.util.zip.ZipException: encrypted ZIP entry not supported', 'Archive contains a file (./../frontend/dist/frontend/assets/public/videos/owasp_promo.vtt) that would be extracted outside of the target directory.', 'Archive contains a file (./../ftp/legal.md) that would be extracted outside of the target directory.', and 'java.io.EOFException: Expected to read bytes from the stream'. The 'Summary' section includes a link to show all vulnerable dependencies. Below this is a table header with columns: Dependency, Vulnerability IDs, Package, Highest Severity, CVE Count, Confidence, and Evidence Count. The 'Dependencies (vulnerable)' section contains a disclaimer about data sources: National Vulnerability Database, CISA Known Exploited Vulnerability Catalog, GitHub Advisory Database (via NPM Audit API), and Snyk Open Source Security API.

Figura 2 Relatório HTML real do OWASP Dependency Check.
Fonte: relatório gerado pelo Grupo 4, 2026.

Por que o resultado é inconclusivo

O log informou que package-lock.json não estava presente e que node_modules não existia em dois projetos Node. A própria ferramenta alertou que essa condição pode produzir falsos negativos. Além disso, o modo --noupdate utilizou uma base previamente armazenada e o OSS Index foi desabilitado. Assim, zero CVEs significa apenas que nenhuma correspondência foi encontrada na cobertura disponível.

Métrica	Resultado	Leitura correta
Dependências identificadas	15	Inventário parcial
Vulnerabilidades	0	Não autoriza concluir que o projeto está seguro
Exceções	4	Arquivos de teste não analisados
Tempo	7 s	Rápido porque não atualizou as bases
Taxa de falso positivo	não calculável	Não houve achado de vulnerabilidade para classificar

Ponto forte: integração madura com builds Java e relatórios amplos. Limitação observada: forte dependência de inventário e bases. Cenário ideal: projeto com lockfiles, dependências resolvidas, cache atualizado e chave NVD. Alternativas incluem Trivy, Gype, OSV-Scanner e ferramentas nativas do ecossistema. O Dependency-Check permanece útil como uma fonte, mas o resultado deve ser combinado com SBOM e outro mecanismo de SCA quando a cobertura do ecossistema for limitada.

IAC SECURITY

6 Checkov

Identificação e atividade do projeto

Atributo	Informação
Mantenedor	Prisma Cloud da Palo Alto Networks e comunidade
Licença	Apache 2.0
Ano de criação	2019
Linguagem principal	Python
Comunidade em 15 set. 2026	aprox. 9,0 mil estrelas, 1,4 mil forks e 17.433 commits
Release atual na consulta	v3.3.17, publicada em 10 set. 2026
Versão do laboratório	v3.3.11

Checkov é um analisador estático de infraestrutura como código mantido pela Prisma Cloud. O projeto informa mais de mil políticas embutidas e suporte a Terraform, planos Terraform, CloudFormation, Kubernetes, Helm, Kustomize, Dockerfile, Bicep, OpenTofu e

arquivos de pipelines [11]. A frequência de releases e o volume de commits mostram atividade contínua; a versão 3.3.17 foi publicada cinco dias antes da consulta [13].

Fundamento técnico

A ferramenta interpreta os arquivos declarativos, cria uma representação dos recursos e aplica políticas Python ou YAML. Algumas verificações examinam atributos locais; outras usam um grafo para avaliar relações, como uma sub-rede ligada a um recurso público. O resultado indica check, recurso, arquivo, linhas, orientação e documentação.

Checkov detecta configurações incompatíveis com boas práticas ou normas codificadas: portas abertas, criptografia ausente, logs desabilitados, containers privilegiados e pipelines permissivas. Ele não prova que o recurso foi implantado, não vê alterações manuais posteriores e pode desconhecer controles externos aplicados pela organização. Variáveis não resolvidas e módulos remotos também afetam precisão.

CHECKOV NA PRÁTICA

Instalação políticas integração e supressões

Instalação e formatos

Checkov pode ser instalado por pip, pipx, Homebrew, Docker e action de CI. O laboratório usou a imagem ghcr.io/bridgecrewio/checkov:3.3.11. A ferramenta gera CLI, JSON, JUnit XML, CSV, SARIF, CycloneDX e Markdown para GitHub, conforme o tipo de scan [11].

```
docker run --rm \
-v "${ProjectDir}target/juice-shop/terraform:/tf:ro" \
ghcr.io/bridgecrewio/checkov:3.3.11 -d /tf \
--framework terraform --quiet --compact --soft-fail
```

Políticas gerais sem regra própria

A execução utilizou somente políticas nativas. Nenhuma regra do grupo foi necessária para encontrar problemas. O argumento `--framework terraform` limitou a tecnologia e `--compact` reduziu o volume. `--soft-fail` fez o processo devolver sucesso para que a saída pudesse ser fotografada e analisada; ele não transformou as 24 falhas em aprovações.

Regras próprias e exceções

Políticas personalizadas podem ser escritas em Python ou YAML e carregadas por diretório. Uma regra pode representar um requisito interno que não existe no catálogo. O

comentário checkov:skip permite suprimir um check em um recurso, e --skip-check exclui IDs. A justificativa deve acompanhar a exceção. A configuração .checkov.yml centraliza frameworks, formatos, diretórios e política de falha.

CI IDE e pre commit

Checkov possui exemplos de GitHub Actions, GitLab e outros sistemas, integração por SARIF com Code Scanning e suporte a pre-commit. A extensão da Prisma Cloud oferece feedback em VS Code. O melhor ponto de execução é antes do merge e novamente sobre o plano final no estágio de deploy, pois variáveis podem mudar o recurso resultante. Resultados SARIF ou JSON podem ser enviados a plataformas de gestão como DefectDojo.

CHECKOV EXECUTADO

Resultado do Terraform oficial do Juice Shop

O scan analisou 26 recursos Terraform e aplicou 66 verificações: 42 passaram, 24 falharam e nenhuma foi ignorada. Não houve erro de parsing. A captura abaixo mostra a execução real e os primeiros controles reprovados.

```
PS C:\Users\Hayom\Documents\CheckpointDevOps> docker run --rm \
>> -v "${ProjectDir}\target\juice-shop\terraform:tf:ro" \
>> ghcr.io/bridgecrewio/checkov:3.3.11 \
>> -d /tf \
>> --framework terraform \
>> --quiet \
>> --compact \
>> --soft-fail 2>&1 |
>> Tee-Object ".\reports\checkov-live.log"
terraform scan results:

Passed checks: 42, Failed checks: 24, Skipped checks: 0

Check: CKV_AWS_336: "Ensure ECS containers are limited to read-only access to root filesystems"
      FAILED for resource: aws_ecs_task_definition.juice_shop
      File: /main.tf:35-97
      Guide: https://docs.prismacloud.io/en/enterprise-edition/policy-reference/aws-policies/aws-general-policies/bc-aws-336
Check: CKV_AWS_333: "Ensure ECS services do not have public IP addresses assigned to them automatically"
      FAILED for resource: aws_ecs_service.juice_shop
      File: /main.tf:99-124
      Guide: https://docs.prismacloud.io/en/enterprise-edition/policy-reference/aws-policies/aws-logging-policies/bc-aws-333
Check: CKV_AWS_338: "Ensure CloudWatch log groups retains logs for at least 1 year"
      FAILED for resource: aws_cloudwatch_log_group.juice_shop
```

Figura 3 Execução real do Checkov no Terraform.
Fonte: captura de terminal do Grupo 4, 2026.

Falhas priorizadas

Check	Recurso	Condição observada	Tratamento
CKV_AWS_333	ECS service	IP público atribuído	Executar em sub-rede privada
CKV_AWS_260	Security group ALB	Porta 80 aberta para 0.0.0.0/0	Restringir ou redirecionar
CKV_AWS_2	ALB listener	Protocolo HTTP	Configurar HTTPS e certificado
CKV_AWS_91	Application Load Balancer	Access logs ausentes	Habilitar logs em bucket protegido
CKV2_AWS_11	VPC	Flow logs ausentes	Habilitar VPC Flow Logs
CKV2_AWS_28	ALB público	WAF ausente	Associar Web ACL

Avaliação crítica

Os seis controles priorizados foram confirmados no Terraform e não foram classificados como falsos positivos na amostra. Os 18 restantes não passaram por validação manual completa, portanto não é correto declarar taxa zero para o conjunto inteiro. O tempo aproximado de nove segundos torna o Checkov adequado a pull requests. O ponto forte é a ampla biblioteca de políticas; a limitação é que uma recomendação genérica pode conflitar com arquitetura, compensações ou módulos externos. Alternativas incluem KICS, Terrascan, tfsec incorporado ao Trivy e Conftest/OPA.

DAST

7 OWASP ZAP

Identificação e atividade do projeto

Atributo	Informação
Mantenedor	ZAP Core Team com apoio da Checkmarx e comunidade
Licença	Apache 2.0
Ano de criação	2010
Linguagem principal	Java
Comunidade em 15 set. 2026	aprox. 15,8 mil estrelas, 2,6 mil forks e 10.387 commits
Release estável atual e executada	v2.17.0, publicada em 15 dez. 2025

Atributo	Informação
Cadência adicional	builds weekly e add-ons atualizados separadamente

ZAP é um proxy de interceptação e scanner de aplicações web mantido pelo ZAP Core Team [14]. Pode ser usado de forma interativa por analistas ou de modo headless em CI. A release 2.17.0 introduziu melhorias de desempenho e redução de alertas duplicados [17]. O ecossistema de add-ons atualiza spiders, regras e integrações sem depender somente de uma nova versão do core.

Fundamento técnico

O DAST observa HTTP e HTTPS sem ler o código-fonte. O spider tradicional segue links e formulários; o AJAX spider controla um navegador para aplicações modernas; o passive scanner analisa mensagens sem alterar requisições; o active scanner envia payloads e procura diferenças compatíveis com vulnerabilidades. O fuzzer permite variar entradas de forma controlada.

ZAP detecta cabeçalhos ausentes, exposição de informações, configurações de cookie, injeções e outras falhas alcançáveis pelas rotas descobertas. Ele não encontra código não exposto, depende de autenticação e navegação adequadas e pode interpretar uma resposta anormal como vulnerabilidade. Rate limits, dados destrutivos e estados de negócio exigem configuração cuidadosa.

ZAP NA PRÁTICA

Automação regras integração e controle de escopo

Formas de instalação

ZAP oferece instaladores, pacotes multiplataforma, Snap, Flatpak e imagens Docker. O laboratório usou ghcr.io/zaproxy/zaproxy:2.17.0. O Automation Framework lê um plano YAML com contexto, spiders, scanners, relatórios e exit status [15]. As saídas preservadas foram HTML, JSON e SARIF.

Plano sem regra de detecção própria

O arquivo security/zap-juice-shop.yaml não cria uma assinatura. Ele define URL, contexto, duração e seleciona duas regras ativas já fornecidas pelo ZAP: 40012 para XSS refletido e 40018 para SQL Injection. Portanto, a detecção é geral e nativa; o YAML apenas torna a execução previsível para o laboratório.

```

jobs:
-
  type: spider
  parameters: { maxDuration: 1 }
-
  type: spiderAjax
  parameters: { maxDuration: 2, maxCrawlDepth: 5 }
-
  type: activeScan
  parameters: { maxScanDurationInMins: 4 }
  policyDefinition:
    rules: [ { id: 40012 }, { id: 40018 } ]

```

Integração e supressões

ZAP pode rodar por baseline scan, full scan, API scan, Automation Framework e GitHub Action. SARIF integra os achados ao Code Scanning, e relatórios ZAP podem ser importados no DefectDojo. A ferramenta não é adequada a pre-commit, pois exige aplicação ativa. Em IDE, o uso é indireto por proxy e testes manuais. Falsos positivos podem ser tratados com alert filters no contexto; a exceção deve indicar URL, regra, evidência e validade.

Segurança operacional

Active scan envia payloads potencialmente destrutivos. O alvo deve ser autorizado, isolado e descartável. Contas de teste e dados sintéticos evitam dano. O plano do grupo usa rede local, limite de tempo e apenas duas regras ativas para caber na apresentação.

OWASP ZAP EXECUTADO

Resultados do scan dinâmico

O spider tradicional encontrou 101 URLs em 18 segundos. O AJAX spider encontrou 234 URLs em 37 segundos. O active scan durou 3 minutos e 48 segundos. O relatório consolidou 11 tipos de alerta em 39 instâncias: 1 High, 4 Medium, 3 Low e 3 Informational. O gate encontrou um bloqueador e rejeitou o cenário.

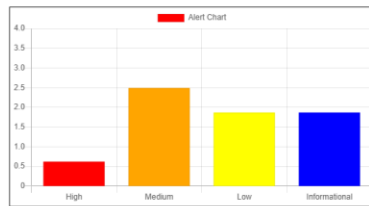


Grupo 4 — OWASP ZAP em Juice Shop v20.2.0

Generated on Wed, 16 Sept 2026 00:46:13

ZAP Version: 2.17.0

ZAP by Checkmarx



Summary of Alerts

Risk Level	Number of Alerts
High	1
Medium	4
Low	3
Informational	3
False Positives	0

Insights

Level	Reason	Site	Description	Statistic
Low	Warning		ZAP errors logged - see the zap.log file for details	281
Low	Warning		ZAP warnings logged - see the zap.log file for details	5
Low	Exceeded Low		Percentage of network failures	5 %
Info	Informational	http://juice-shop:3000	Percentage of responses with status code 1xx	1 %
Info	Informational	http://juice-shop:3000	Percentage of responses with status code 2xx	55 %
Info	Informational	http://juice-shop:3000	Percentage of responses with status code 3xx	28 %
Info	Informational	http://juice-shop:3000	Percentage of responses with status code 4xx	14 %
Info	Informational	http://juice-shop:3000	Percentage of responses with status code 5xx	1 %
Info	Informational	http://juice-shop:3000	Percentage of endpoints with content type application/javascript	8 %
Info	Informational	http://juice-shop:3000	Percentage of endpoints with content type application/json	4 %
Info	Informational	http://juice-shop:3000	Percentage of endpoints with content type application/octet-stream	8 %
Info	Informational	http://juice-shop:3000	Percentage of endpoints with content type font/woff2	1 %
Info	Informational	http://juice-shop:3000	Percentage of endpoints with content type image/jpeg	11 %
Info	Informational	http://juice-shop:3000	Percentage of endpoints with content type image/png	5 %
Info	Informational	http://juice-shop:3000	Percentage of endpoints with content type image/x-icon	1 %

Figura 4 Resumo HTML real do OWASP ZAP.
Fonte: relatório gerado pelo Grupo 4, 2026.

Risco	Plugin	Alerta	Instâncias	Confiança
High	40018	SQL Injection	1	Low
Medium	10038	CSP Header Not Set	5	High
Medium	10098	Cross-Domain Misconfiguration	5	Medium
Medium	10020	Missing Anti-clickjacking Header	3	Medium
Medium	3	Session ID in URL Rewrite	5	High

A confiança baixa do alerta SQL Injection impede tratá-lo isoladamente como prova. A correlação com o Sengrep mostra o mesmo parâmetro chegando a uma consulta interpolada e sustenta a classificação como verdadeiro positivo. Entre os três alertas revisados manualmente, nenhum foi descartado como falso positivo; os oito restantes não foram integralmente reproduzidos. O principal ponto forte do ZAP é observar o sistema montado. As limitações são tempo, cobertura de navegação, autenticação e ruído. Alternativas incluem Wapiti, Nuclei e Nikto, com escopos e técnicas diferentes.

VERDADEIROS E FALSOS POSITIVOS

8 Análise dos achados

Achado 1 SQL Injection na busca

Campo	Análise
Evidência	Semgrep em routes/search.ts:23 e ZAP plugin 40018 no parâmetro q
Classificação	Verdadeiro positivo
CWE	CWE-89 Improper Neutralization of Special Elements used in an SQL Command
Impacto	Alteração da lógica da consulta e possível acesso indevido a dados
Correção	Consulta parametrizada com :criteria e replacements; menor privilégio no banco
Validação	Semgrep green sem achados no arquivo corrigido; novo DAST recomendado

O código concatenava criteria dentro de um template SQL. O ZAP enviou um payload ao endpoint e observou resposta compatível com injeção, porém marcou confiança baixa. A evidência estática remove a principal ambiguidade do alerta dinâmico. A correção aplicada segue o princípio de separar dados e comando; escape manual ou blacklist de caracteres não oferecem a mesma garantia [4][19].

Achado 2 listener HTTP no load balancer

Campo	Análise
Evidência	Checkov CKV_AWS_2 em aws_lb_listener.http
Classificação	Verdadeiro positivo
CWE	CWE-319 Cleartext Transmission of Sensitive Information
Impacto	Tráfego e credenciais podem ser expostos ou alterados em trânsito
Correção	Listener HTTPS com certificado; redirecionamento permanente de HTTP para HTTPS
Validação	Reexecutar Checkov no plano e testar redirecionamento no ambiente

O Terraform define um listener HTTP e um target group HTTP. Como o recurso está voltado para exposição pública, a ausência de TLS no frontend não é apenas uma preferência de política. A correção deve incluir certificado válido e redirecionamento, além de confirmar se o tráfego interno também precisa de criptografia.

TRIAGEM DE CONTEXTO

Achado 3 ocorrência em snippet didático

Campo	Análise
Evidência	Regra comunitária do Semgrep em data/static/codefixes/unionSqlInjectionChallenge_1.ts
Classificação	Falso positivo operacional para o código implantado
CWE	CWE-89 no padrão sintático encontrado
Justificativa	O arquivo é um exemplo didático armazenado como conteúdo de desafio, não uma rota executada pelo servidor
Tratamento	Excluir o caminho didático do gate, sem remover a regra das rotas reais
Validação	Confirmar no build que o arquivo não compõe o bundle executável

A regra encontrou corretamente uma construção insegura, mas o contexto muda a decisão. Classificar o item como falso positivo operacional não significa que a regra está errada; significa que o arquivo não representa uma superfície executável no alvo. Uma exclusão precisa ser específica para data/static/codefixes. Ignorar toda ocorrência de SQL Injection ocultaria routes/search.ts e routes/login.ts.

Resumo da triagem

Achado	Ferramenta	VP ou FP	Prioridade	Próxima ação
SQL Injection na busca	Semgrep e ZAP	VP	Alta	Parametrizar e retestar
Listener HTTP	Checkov	VP	Alta	HTTPS e redirecionamento
SQLi em codefix didático	Semgrep	FP operacional	Baixa	Excluir somente o caminho

Critério de análise

A classificação considerou existência da condição, alcance no artefato implantado, evidência independente e possibilidade de reprodução. O mesmo alerta pode mudar de prioridade quando a exposição, o dado processado ou um controle compensatório muda. Por isso, a triagem deve permanecer associada à versão do código e do ambiente.

AVALIAÇÃO TRANSVERSAL

9 Comparação e escolha do toolchain

Critério	Semgrep	Dependency Check	Checkov	OWASP ZAP
Entrada	Código	Dependências	IaC	Aplicação ativa
Técnica	AST, padrões e taint	Evidência, CPE e CVE	Políticas e grafo	Spider, proxy e payloads
Velocidade no laboratório	~3 min geral	7 s	~9 s	~4 min 48 s
Formato principal	JSON e SARIF	HTML, JSON e SARIF	CLI, JSON e SARIF	HTML, JSON e SARIF
Melhor estágio	Code e Build	Build	Code e Deploy	Test e Release
Limitação observada	Ruído em snippets	Inventário incompleto	Política sem contexto	Cobertura e confiança
Resultado	69 geral; red 1 e green 0	0 CVEs, inconclusivo	42 pass e 24 fail	11 tipos; 1 High

Recomendação do Grupo 4

O toolchain recomendado mantém as quatro ferramentas. Semgrep e Checkov executam em pull requests por oferecerem feedback rápido. Dependency-Check executa após a resolução das dependências, com cache atualizado e lockfiles presentes. ZAP roda contra um ambiente efêmero após o deploy de teste. Um job agendado pode ampliar regras e tempo sem atrasar todo commit.

Momento	Controle	Gate recomendado
Pre-commit e IDE	Semgrep rápido e Checkov nos arquivos alterados	Aviso local; sem depender de rede
Pull request	Semgrep e Checkov completos	Bloquear novos HIGH e CRITICAL
Build	Dependency-Check com lockfile e base atualizada	CVSS, alcance e exceções válidas
Ambiente de teste	ZAP autenticado e limitado ao alvo	Risco, confiança e reprodução
Agendado	Scans amplos das quatro ferramentas	Abrir backlog e revisar baselines

A principal razão para manter ferramentas especializadas é preservar a pergunta de cada controle. Um resultado unificado ajuda a priorizar, mas não deve reduzir SCA, IaC e DAST a um único número. O relatório original continua necessário para entender cobertura e corrigir o problema certo.

BUILD RED E BUILD GREEN

10 Integração contínua e quality gate

O workflow `.github/workflows/devsecops.yml` contém dois jobs independentes. O primeiro executa o Semsigrep no cenário vulnerável, confirma que o gate bloqueia e depois repete sobre a cópia corrigida. O segundo sobe o Juice Shop local, executa o ZAP e confirma o bloqueio. Os relatórios são publicados com `if: always` para permanecerem disponíveis mesmo quando a decisão de segurança é negativa.

Cenário	Relatório	Bloqueadores	Resultado esperado
SAST red	<code>semsigrep-red.json</code>	1 HIGH	Teste comprova gate bloqueado
SAST green	<code>semsigrep-green.json</code>	0	Gate aprovado
DAST	<code>zap-juice-shop.json</code>	1 HIGH	Teste comprova gate bloqueado

Normalização

O script `scripts/quality-gate.mjs` converte severidades do Semsigrep e `riskcode` do ZAP para LOW, MEDIUM, HIGH e CRITICAL. O limiar padrão é HIGH. No modo `expect fail`, o teste termina com sucesso somente quando encontra um bloqueador; isso permite demonstrar o build vermelho sem transformar a validação do laboratório em falha acidental do workflow.

Ampliação para SCA e IaC

Checkov pode entrar em um terceiro job e remover `--soft-fail` quando a política estiver acordada. Dependency-Check deve ser adicionado depois da geração do lockfile e instalação das dependências, com dados do NVD em cache. Adicionar o SCA antes de corrigir a cobertura apenas automatizaria um falso sentimento de segurança.

Governança do gate

- Bloquear achados novos acima do limiar e preservar o relatório completo.
- Exigir responsável, motivo e data de expiração para exceções.
- Distinguir erro da ferramenta, falta de cobertura e ausência real de achados.
- Medir idade, reincidência, tempo de correção, cobertura e falsos positivos triados.
- Reavaliar baselines quando a regra, a versão da ferramenta ou a exposição mudar.

A política técnica só funciona quando existe um caminho de correção. Um gate que bloqueia sem publicar localização, evidência e orientação transfere o problema para o fim do processo. O desenho do grupo mantém JSON para decisão automática, SARIF para revisão e HTML para leitura e plano B.

SÍNTESE CRÍTICA

11 Limitações e conclusão

Limitações do experimento

- O Semgrep geral gerou 69 achados, mas somente recortes prioritários foram triados manualmente.
- O red e green do Semgrep analisou um arquivo para garantir execução previsível durante a apresentação.
- O Dependency-Check não recebeu lockfiles nem node_modules e foi executado sem atualizar a base.
- O Checkov avaliou o Terraform oficial, não um plano resolvido nem recursos já implantados.
- O ZAP não autenticou usuários e limitou o active scan a quatro minutos e duas regras ativas.
- A imagem oficial do Juice Shop permaneceu vulnerável; o green foi uma cópia usada apenas pelo SAST.

Conclusão

As quatro ferramentas foram executadas e produziram resultados distintos. O Semgrep ofereceu ampla visibilidade do código e comprovou uma correção específica. O Dependency-Check mostrou, pela ausência de inventário suficiente, que um scan concluído pode continuar inconclusivo. O Checkov revelou 24 desvios concretos no Terraform. O ZAP observou o sistema em execução e encontrou um alerta HIGH correlacionável ao código.

A correlação mais forte foi a SQL Injection da busca. Semgrep mostrou a interpolação na consulta; ZAP mostrou o endpoint respondendo de forma compatível com um payload de injeção; a consulta parametrizada removeu o padrão no cenário green. Essa sequência fornece evidência de achado, correção e verificação, sem afirmar que um único teste tornou a aplicação segura.

O Grupo 4 recomenda executar Semgrep e Checkov cedo, Dependency-Check com inventário completo no build e ZAP em um ambiente local ou efêmero de teste. Gates devem tratar HIGH e CRITICAL, mas também precisam registrar cobertura, confiança e exceções. A combinação oferece defesa em profundidade porque cada ferramenta enxerga uma parte que as outras não observam.

Orientação para a apresentação

Na apresentação, mostrar o Semgrep geral, o red e green da consulta e o relatório do ZAP. Checkov entra com as 24 falhas e Dependency-Check como exemplo de cobertura insuficiente. Essa leitura demonstra domínio técnico além das contagens.

FONTES PRIMÁRIAS E NORMAS

Referências

- [1] NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. Secure Software Development Framework SSDF Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities. Gaithersburg, 2022. Disponível em: <https://csrc.nist.gov/pubs/sp/800/218/final>. Acesso em: 15 set. 2026.
- [2] OWASP FOUNDATION. DevSecOps Guideline: Vulnerability Scanning. Disponível em: <https://owasp.org/www-project-devsecops-guideline/latest/02-Vulnerability-Scanning.html>. Acesso em: 15 set. 2026.
- [3] CYBERSECURITY AND INFRASTRUCTURE SECURITY AGENCY. 2025 Minimum Elements for a Software Bill of Materials. Washington, 2025. Disponível em: <https://www.cisa.gov/resources-tools/resources/2025-minimum-elements-software-bill-materials-sbom>. Acesso em: 15 set. 2026.
- [4] MITRE. CWE-89: Improper Neutralization of Special Elements used in an SQL Command. Disponível em: <https://cwe.mitre.org/data/definitions/89.html>. Acesso em: 15 set. 2026.
- [5] SEMGREP. Semgrep repository. GitHub, 2026. Disponível em: <https://github.com/semgrep/semgrep>. Acesso em: 15 set. 2026.
- [6] SEMGREP. Continuous integration documentation. Disponível em: <https://semgrep.dev/docs/semgrep-ci/overview>. Acesso em: 15 set. 2026.
- [7] SEMGREP. Release v1.177.0. GitHub, 10 set. 2026. Disponível em: <https://github.com/semgrep/semgrep/releases/tag/v1.177.0>. Acesso em: 15 set. 2026.
- [8] OWASP DEPENDENCY-CHECK. Official repository and usage documentation. GitHub, 2026. Disponível em: <https://github.com/dependency-check/DependencyCheck>. Acesso em: 15 set. 2026.
- [9] OWASP DEPENDENCY-CHECK. Command Line Arguments. Disponível em: <https://dependency-check.github.io/DependencyCheck/dependency-check-cli/arguments.html>. Acesso em: 15 set. 2026.
- [10] OWASP DEPENDENCY-CHECK. Version 13.0.0. GitHub, 3 ago. 2026. Disponível em: <https://github.com/dependency-check/DependencyCheck/releases/tag/v13.0.0>. Acesso em: 15 set. 2026.
- [11] BRIDGECREW. Checkov repository. GitHub, 2026. Disponível em: <https://github.com/bridgecrewio/checkov>. Acesso em: 15 set. 2026.
- [12] CHECKOV. Output formats. Disponível em: <https://www.checkov.io/8.Outputs/Output%20Formats.html>. Acesso em: 15 set. 2026.
- [13] BRIDGECREW. Checkov release 3.3.17. GitHub, 10 set. 2026. Disponível em: <https://github.com/bridgecrewio/checkov/releases/tag/3.3.17>. Acesso em: 15 set. 2026.
- [14] ZAPROXY. ZAP core repository. GitHub, 2026. Disponível em: <https://github.com/zaproxy/zaproxy>. Acesso em: 15 set. 2026.
- [15] ZAPROXY. Automation Framework. Disponível em: <https://www.zaproxy.org/docs/automate/automation-framework/>. Acesso em: 15 set. 2026.
- [16] ZAPROXY. ZAP Docker images. Disponível em: <https://www.zaproxy.org/docs/docker/about/>. Acesso em: 15 set. 2026.
- [17] ZAPROXY. ZAP 2.17.0. 15 dez. 2025. Disponível em: <https://www.zaproxy.org/blog/2025-12-15-zap-2-17-0/>. Acesso em: 15 set. 2026.
- [18] OWASP JUICE SHOP. Release v20.2.0. GitHub, 2026. Disponível em: <https://github.com/juice-shop/juice-shop/releases/tag/v20.2.0>. Acesso em: 15 set. 2026.
- [19] OWASP FOUNDATION. OWASP Top 10: A03 Injection. Disponível em: https://owasp.org/Top10/A03_2021-Injection/. Acesso em: 15 set. 2026.
- [20] GITHUB. Uploading a SARIF file to GitHub. Disponível em: <https://docs.github.com/code-security/code-scanning/integrating-with-code-scanning/uploading-a-sarif-file-to-github>. Acesso em: 15 set. 2026.
- [21] OWASP FOUNDATION. CycloneDX. Disponível em: <https://owasp.org/www-project-cyclonedx/>. Acesso em: 15 set. 2026.

REPRODUÇÃO

Apêndice A Comandos principais

Preparação

Set-ExecutionPolicy	-Scope	Process	Bypass
Set-Location		"C:\Users\Hayom\Documents\CheckpointDevOps"	
docker version			

Laboratório red green e DAST

.\scripts\run-lab.ps1	-Mode	sast-red
.\scripts\run-lab.ps1	-Mode	sast-green
.\scripts\run-lab.ps1 -Mode dast		

Execuções complementares

```
#
docker run --rm -v "${ProjectDir}target/juice-shop:/src/target/juice-shop" \
  semgrep/semgrep:1.172.0 semgrep scan --config auto /src/target/juice-shop
#
docker run --rm -v "${ProjectDir}target/juice-shop/terraform:/tf:ro" \
  ghcr.io/bridgecrewio/checkov:3.3.11 -d /tf --framework terraform --compact
```

Encerramento

```
docker compose --profile tools down --remove-orphans
```

Os comandos completos, resultados esperados e solução de problemas estão em LAB.md. As imagens Docker permanecem em cache após o encerramento para reduzir o risco de rede durante a apresentação.

RASTREABILIDADE

Apêndice B Mapa de evidências

Evidência	Arquivo	Finalidade
Semgrep geral	reports/semgrep-auto.json e semgrep-auto.log	Cobertura geral sem regra própria
Semgrep red	reports/semgrep-red.json	Comprovar HIGH e bloqueio
Semgrep green	reports/semgrep-green.json	Comprovar ausência do padrão após correção
Dependency-Check	reports/dependency-check/	HTML, JSON, XML, CSV e SARIF
Checkov	reports/checkov.json e checkov.sarif	42 aprovados e 24 reprovados
OWASP ZAP	reports/zap-juice-shop.html e .json	11 tipos de alerta e gate
Captura Semgrep	evidencias/real-semgrep-terminal.png	Execução real no terminal
Captura Checkov	evidencias/real-checkov-terminal.png	Execução real no terminal
Captura Dependency-Check	evidencias/real-dependency-check.png	Relatório HTML real
Captura ZAP	evidencias/real-zap-resumo.png	Relatório HTML real

Perguntas de verificação para a turma

1. Quantos achados HIGH o cenário Semgrep red produz e qual é o resultado após a correção?
2. Qual plugin do ZAP reporta a SQL Injection e qual CWE está associado ao alerta?

TRANSPARÊNCIA ACADÊMICA

Apêndice C Uso de inteligência artificial

Ferramentas de inteligência artificial apoiaram a organização inicial do repositório, a redação, a revisão do relatório e a preparação de scripts. O grupo validou as afirmações por meio dos relatórios produzidos pelas ferramentas, do código versionado e das documentações oficiais citadas.

As decisões de escopo, execução, classificação de achados e apresentação permanecem sob responsabilidade do Grupo 4. Métricas como quantidade de arquivos, regras, checks, URLs, alertas, versões e tempos foram conferidas nos artefatos gerados em 15 de setembro de 2026. A declaração detalhada está em USO-DE-IA.md.

Checklist de entrega relacionado ao relatório

Item	Situação	Evidência
Quatro ferramentas cobertas	Atendido	Seções 4 a 7
Execução das quatro ferramentas	Atendido	reports e figuras 1 a 4
Duas categorias no laboratório	Atendido	Semgrep e ZAP
Build red e green	Atendido	semgrep-red.json e semgrep-green.json
Três achados classificados	Atendido	Seção 8
Mínimo de dez fontes	Atendido	21 referências, maioria primária
Plano B	Atendido	Relatórios e capturas versionados
Declaração de IA	Atendido	USO-DE-IA.md e este apêndice